01 Project overview and architecture

AIMLCZG523 | MLOps Assignment 01 | Student ID: 2025ae05588

Heart disease classification: reproducible model development, experiment tracking, tested serving, local Kubernetes deployment and operational monitoring. Prepared 20 October 2026.

The UCI Cleveland labels are predicted from 13 health-data features. This is an educational classifier of a recorded diagnosis label, not a clinically validated diagnosis or future-risk score.

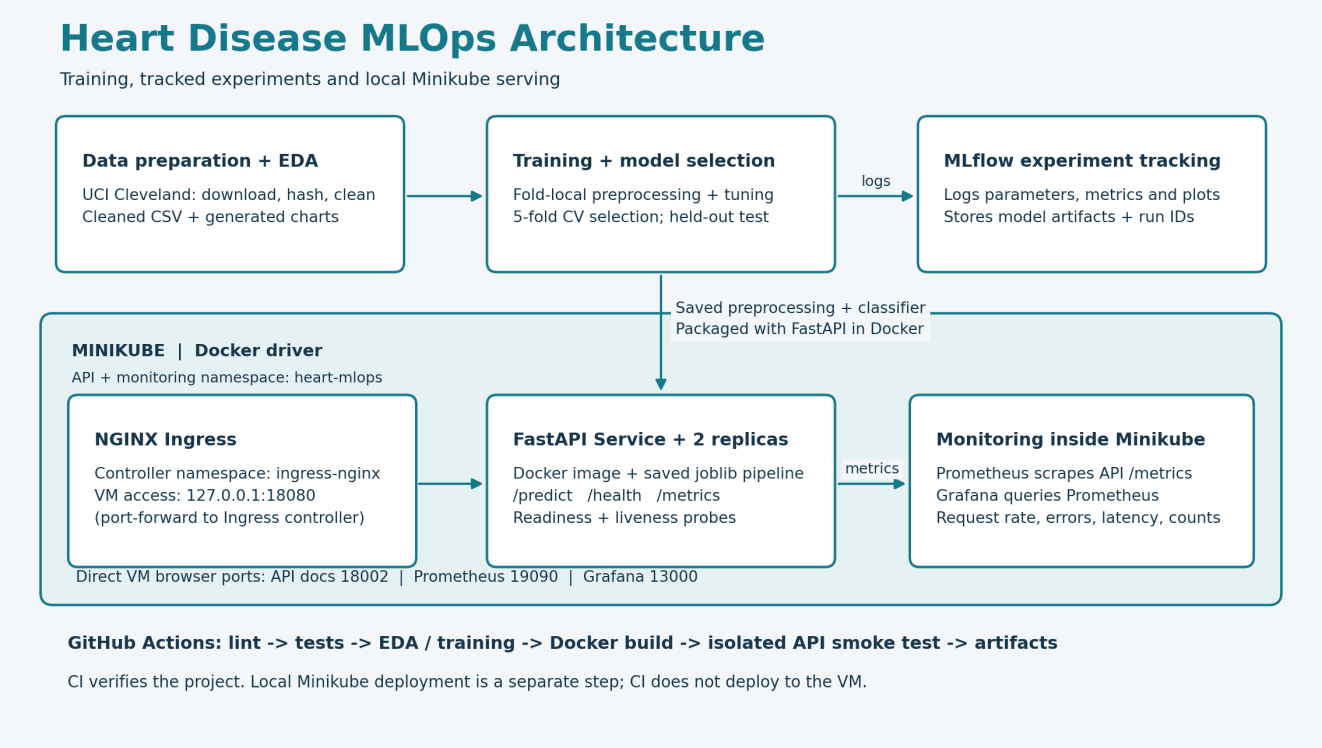


Figure 1. Current Minikube architecture. Training selects the model; MLflow records experiments. NGINX Ingress, API replicas and monitoring run inside Minikube. GitHub Actions verifies the project; local deployment is separate.

GitHub repository - open source, artifacts and instructions

Demonstration video - open the MP4 on GitHub

https://github.com/2025ae05588/ml-assignment1

https://github.com/2025ae05588/ml-assignment1/blob/main/videos/MLOps_Assignment1_2025ae05588_New.mp4

Evidence scope: the submitted model and VM screenshots use MLflow run cab705edb7524048a8ad5b31be8316c4. Later CI runs are independent verification executions. The video is a recorded Prayogshala walkthrough, approximately 10 minutes 21 seconds.

02 Dataset acquisition and preparation

Source: UCI Heart Disease dataset

Citation: Janosi, Steinbrunn, Pfisterer and Detrano (1989), Heart Disease, UCI Machine Learning Repository. DOI: 10.24432/C52P4X. License: CC BY 4.0.

The Cleveland subset contains 14 columns: 13 predictors and the original target. No artificial features were added to satisfy the statement's approximate feature count. scripts/download_data.py downloads the raw file if absent, validates it, writes the cleaned CSV and records provenance in data/metadata.json.

Property	Observed result
Raw / cleaned rows	303 / 303
Exact duplicates removed	0
Missing values	ca: 4; thal: 2; all other predictors: 0
Label mapping	num = 0 -> absent; num = 1, 2, 3, 4 -> present
Class counts	Absent: 164 (54.1%); present: 139 (45.9%)

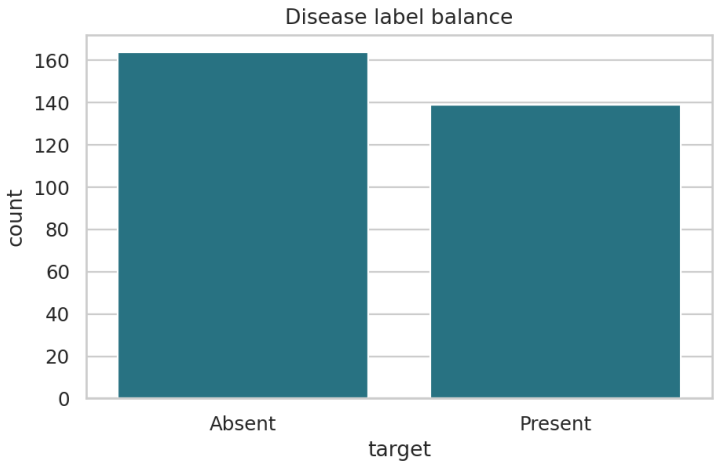


Figure 2. Neither class overwhelmingly dominates; precision, recall and ROC-AUC remain necessary to interpret different classification errors.

Raw-data SHA-256: a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8

Cleaned-data SHA-256: e81973d3356c231ed8903aab16f5faa72d9679eb8ef6dc98a602b25df3f5d23d

Deterministic cleaning preserves missing predictors for fold-local imputation. Original UCI category codes are validated, including cp 1-4, slope 1-3, ca 0-3 and thal 3/6/7.

03 Exploratory data analysis

scripts/eda.py and notebooks/01_eda.ipynb provide reproducible exploration. Histograms inspect numerical feature ranges and distributions; the correlation heatmap summarizes associations. summary.csv and missing.csv preserve descriptive statistics and missing-value counts alongside the PNG figures.

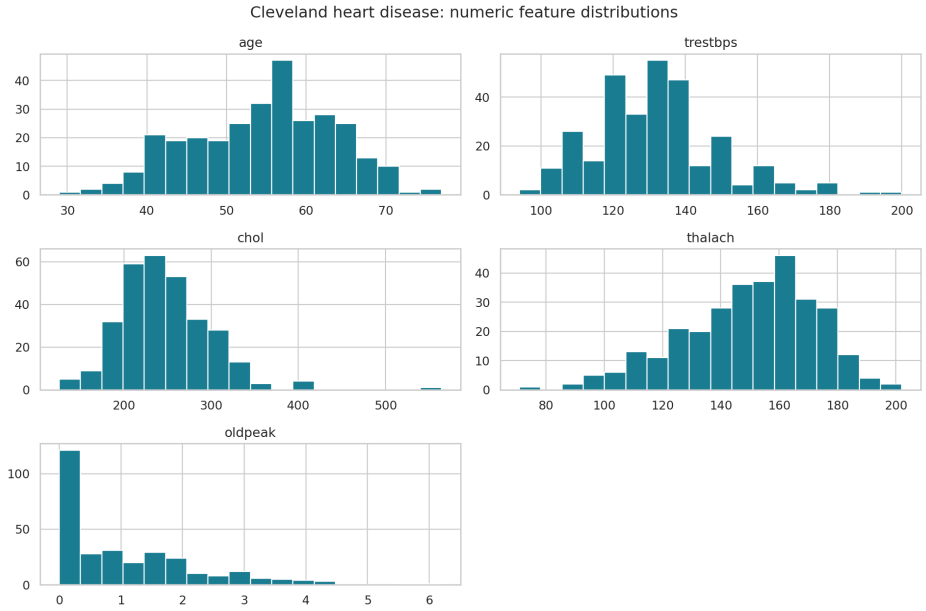


Figure 3. Numerical predictor distributions. Distribution shape and unusual values motivate checking data quality, rather than deleting observations automatically.

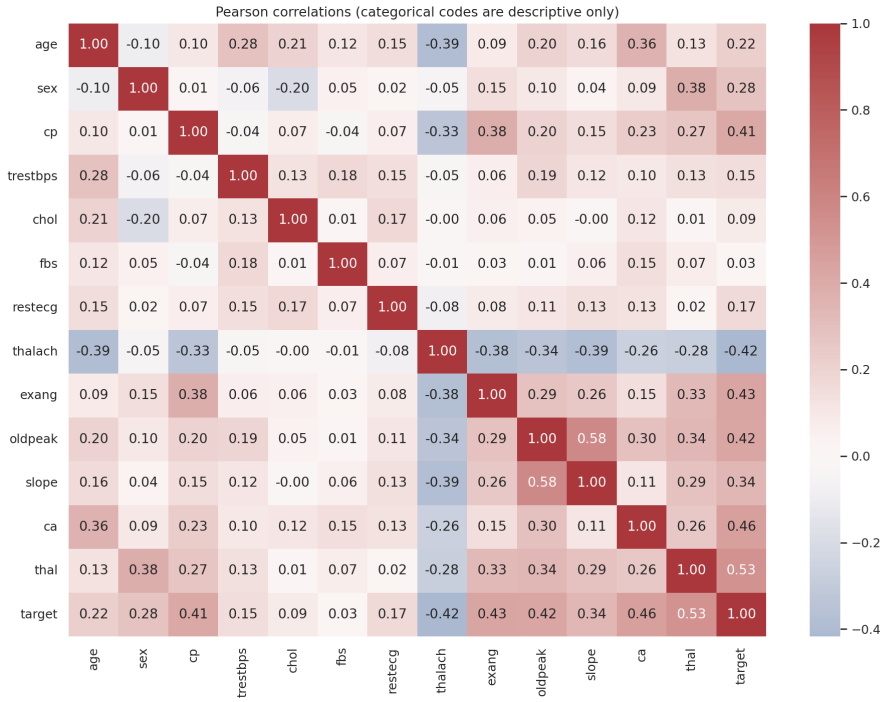


Figure 4. Pearson correlations, including numerically coded categories. Category-code correlations are descriptive; they are not continuous clinical effects or causal claims.

All 13 predictors are retained. The test set is not used to select features. No global scaling or imputation is fitted during EDA; learned transformations remain inside the model pipeline.

04 Feature pipeline and model selection

A single sklearn Pipeline combines a ColumnTransformer and classifier. Numerical features (age, trestbps, chol, thalach, oldpeak) use median imputation and StandardScaler. The eight remaining predictors use most-frequent imputation and OneHotEncoder with handle_unknown='ignore'. Original category values are still validated at the API boundary.

The split is stratified: 242 training records and 61 held-out records, with random seed 42. Five-fold shuffled StratifiedKFold operates only on the training partition. Keeping preprocessing inside the pipeline prevents fold leakage.

Model	Search space / configuration
Logistic Regression	C in {0.1, 1, 10}; max_iter=2000; seed=42
Random Forest	150 trees; max_depth in {3, None}; min_samples_leaf in {1, 4}; seed=42
Comparison	7 configurations, 5 folds; common preprocessing and scoring
Selection	Highest mean CV ROC-AUC; held-out set excluded from selection

Best configuration from each estimator

Estimator	Accuracy	Precision	Recall	ROC-AUC
logistic regression	0.8471	0.8768	0.7834	0.9025
random forest	0.8138	0.8348	0.7462	0.8992

Logistic Regression was selected with C=1.0. The best Random Forest configuration used max_depth=3 and min_samples_leaf=1. The simpler estimator had slightly higher mean CV ROC-AUC; complexity alone was not a reason to prefer the forest.

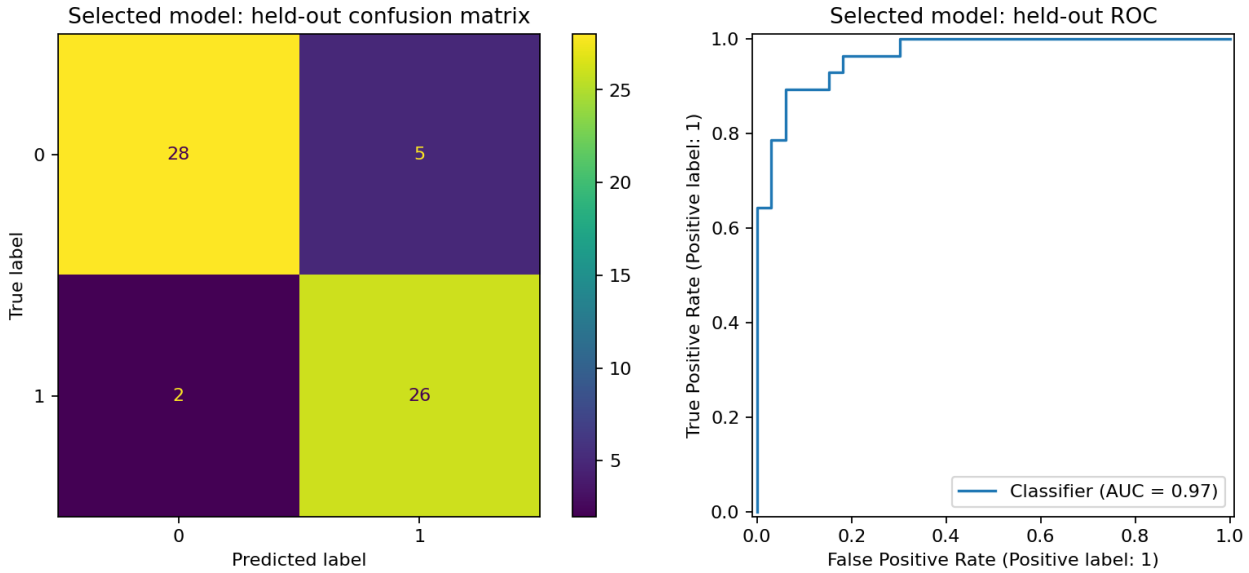
Accuracy, precision, recall and ROC-AUC are recorded for every candidate, including fold means and standard deviations. Full GridSearchCV outputs are stored in artifacts/logistic_regression_cv.csv and artifacts/random_forest_cv.csv.

The exact split indices are preserved in artifacts/split.json. Failed CV fits raise errors rather than being silently substituted with a passing result.

05 Held-out evaluation and limitations

The selected training-only model was evaluated once on the untouched test partition. The decision threshold remained 0.5; no threshold search used the test outcomes.

Held-out metric	Value	Interpretation
Accuracy	0.8852	Fraction of all predictions correct
Precision	0.8387	Correct positives among positive predictions
Recall	0.9286	Detected positives among actual positives
ROC-AUC	0.9665	Discrimination across decision thresholds



Figures 5-6. Held-out confusion matrix and ROC curve for the selected pipeline.

The observed confusion matrix contains 28 true negatives, 26 true positives, 5 false positives and 2 false negatives. Recall is high on this split, but the 61-row test set gives substantial sampling uncertainty.

This small historical sample does not establish generalization to other hospitals or populations. Probabilities have not been externally validated or calibrated for clinical use. Correlations are not causal effects, and the label is not a future-risk outcome.

Future evaluation should include external validation, calibration, subgroup analysis and sensitivity to splits and thresholds. The model is retained as evaluated, without refitting on held-out records.

06 Experiment tracking and model packaging

MLflow uses a local SQLite store (mlflow.db) and local artifact storage. Seven candidate runs log parameters, seed, dataset hash, CV metric means and standard deviations, CV tables and EDA plots. A selected-model-heldout-evaluation run records test metrics, evaluation plots and the full sklearn pipeline.

Submitted VM evaluation run: cab705edb7524048a8ad5b31be8316c4

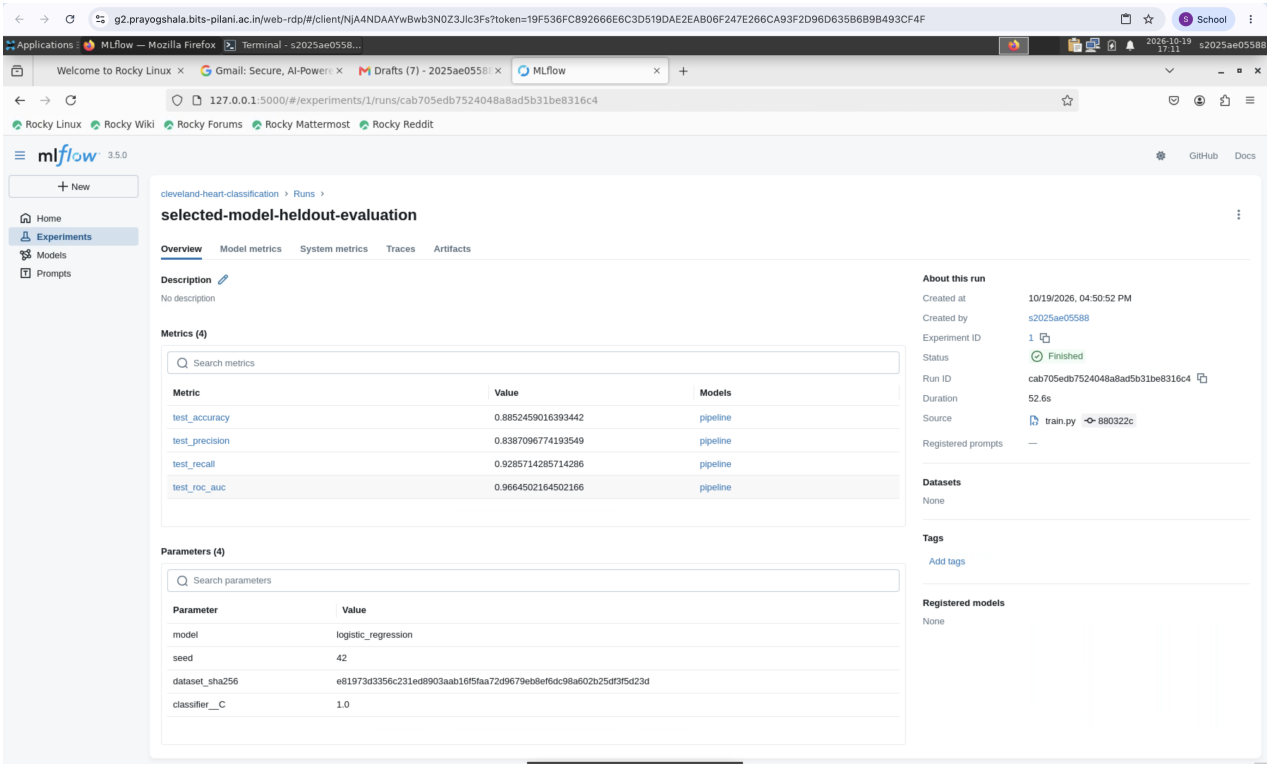


Figure 7. Actual Prayogshala MLflow evaluation run. Its run ID and displayed metrics match artifacts/metrics.json and the API screenshots.

artifacts/model.joblib bundles the fitted preprocessing and classifier. artifacts/metrics.json holds the model version, metrics and data hash. Serving loads this pipeline once at startup instead of reimplementing preprocessing.

requirements.txt pins direct dependencies; requirements-lock.txt records an earlier resolved environment and is a reference snapshot, not a claim that all platforms are identical. Python 3.12 or 3.13 is recommended; CI and Docker use Python 3.13.

MLflow histories are not committed. Re-running training creates new local runs; CI uploads its tracking database and artifacts. The saved VM model remains independently available in Git. Use joblib artifacts only from trusted sources.

```
mlflow ui --backend-store-uri sqlite:///mlflow.db \
  --host 127.0.0.1 --port 5000
```

07 Inference API and Docker isolation

FastAPI provides POST /predict, GET /health, GET /metrics and interactive /docs. Pydantic checks required fields, category codes, numeric bounds, extra fields and non-finite values. ca and thal accept explicit nulls for pipeline imputation.

POST /predict returns prediction, disease_probability, confidence, model_version and an educational-use note. Confidence is the probability assigned to the predicted class, not calibrated clinical certainty.

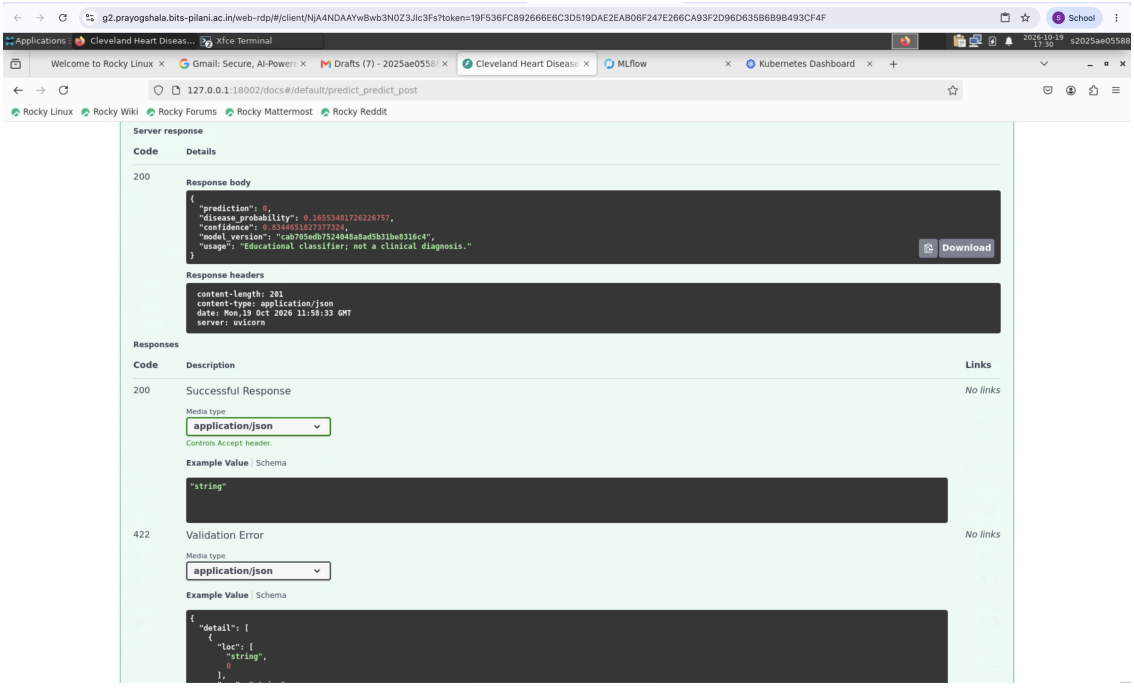


Figure 8. Actual HTTP 200 response from the VM API. The model version matches the submitted MLflow run. The example returns class 0 with confidence about 0.8345.

Docker uses python:3.13-slim, installs the project, copies model.joblib and metrics.json, runs as non-root UID 10001, and serves on port 8000. Its healthcheck calls /health. The model is loaded at application startup; a missing model causes startup failure.

scripts/docker_verify.sh builds the image, starts a temporary isolated container on host port 8001, checks health and JSON prediction, saves logs, then removes that container. The recorded VM build and smoke-test evidence is in screenshots/docker-build-vm.png.

Structured request logs record method, bounded route, status and duration, excluding patient payloads. This reduces unnecessary sensitive-data logging while supporting operational investigation.

```
curl -fsS http://127.0.0.1:18080/predict \
  -H 'Content-Type: application/json' \
  --data @examples/patient.json
```

08 Automated tests and continuous integration

The original VM capture shows 11 passing tests. The repository now has 12 tests, including a deployment regression test added after the capture. Later successful CI runs verify the updated repository without replacing the submitted VM model.

Area	Checks implemented
Data	Target mapping, missing-value preservation, duplicate handling, invalid targets/categories
Model	Training-only imputation statistics; serialized pipeline prediction equivalence
API	Valid prediction, health, metrics, invalid/missing/extra fields and nullable features
Deployment	Mocked command order, content-tagged image, admission readiness and retry

GitHub Actions installs dependencies, runs Ruff, prepares data, executes Pytest, generates EDA, trains, builds Docker, checks /health and /predict, and uploads artifacts even if an earlier stage fails. Failed lint/tests/training stop subsequent normal steps. No cluster credentials or automatic VM deployment are configured in CI.

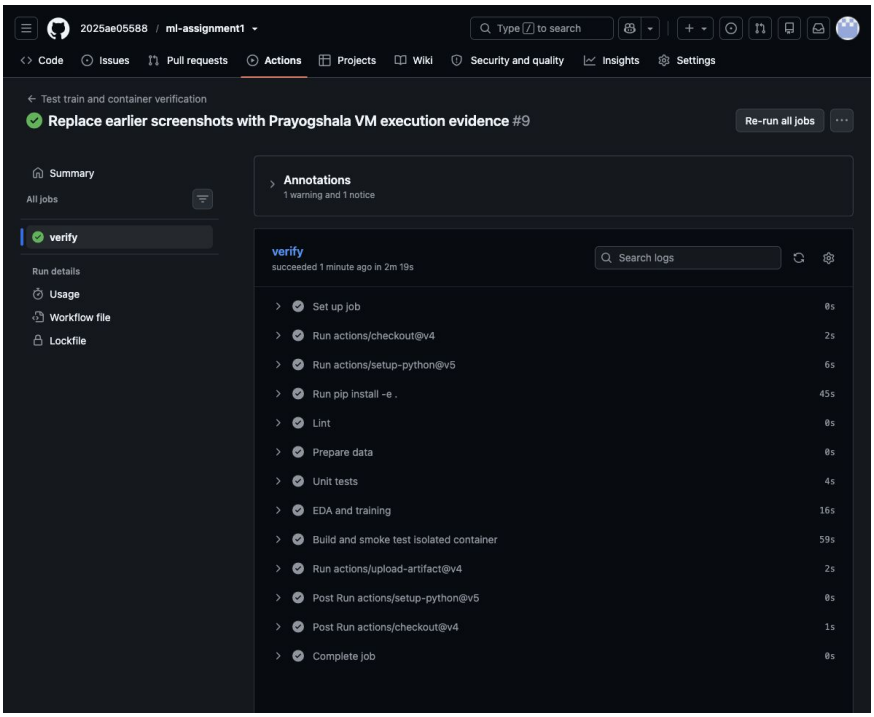


Figure 9. Successful run 39255010564 for screenshot commit 3042161. The screenshot is remote CI evidence, not a VM-local workflow execution.

Later verified CI: architecture commit 618b99e - successful run 39379904180

The workflow uploads model/results/tracking artifacts. CI-generated models have their own run IDs and do not automatically overwrite the committed VM model.

09 Minikube deployment and monitoring

scripts/deploy_local.sh starts profile heart-prayogshala with the Docker driver. It loads a content-tagged image, applies a temporary API manifest, waits for NGINX and admission jobs, retries transient Ingress failures and waits for ready workloads. The committed manifest is unchanged. API and monitoring use namespace heart-mlops; the NGINX controller uses ingress-nginx.

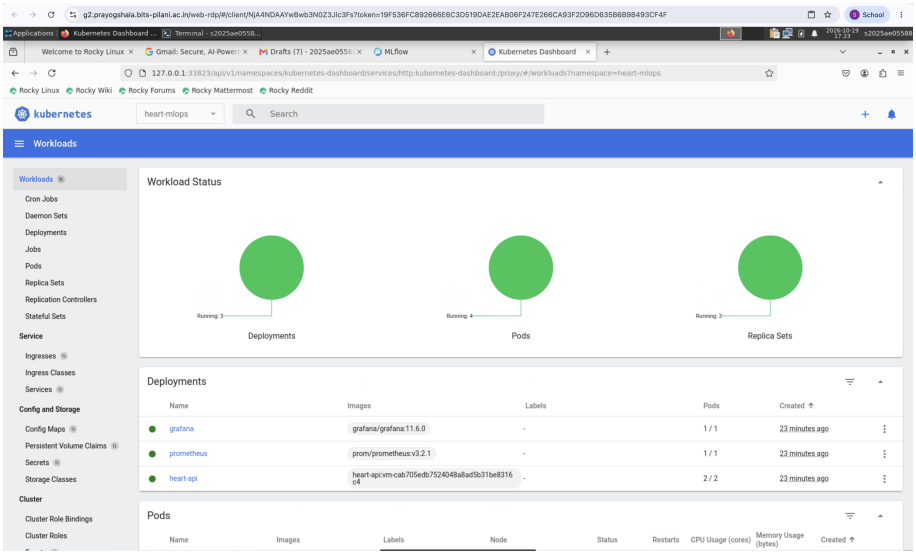


Figure 10. Actual Minikube Dashboard: two API replicas and ready monitoring workloads. Readiness/liveness probes, resource limits, non-root API execution and dropped capabilities are configured in deployment/kubernetes.yaml.

Prometheus scrapes the API Service every five seconds. Grafana is provisioned with the Prometheus datasource and Heart API Operations dashboard. Both run inside Minikube. Traffic through Ingress reaches the API replicas; a direct Service port-forward can select a single pod. Scraping through a load-balanced Service does not preserve per-pod counter identity; pod-level discovery is a production improvement.

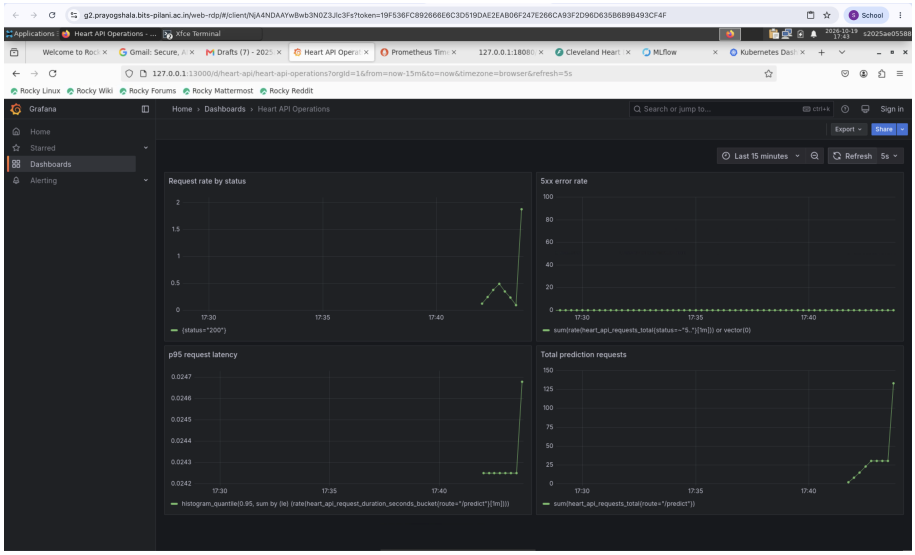


Figure 11. VM request rate, 5xx errors, p95 latency and prediction counts after traffic generation. The Prometheus target-UP capture is also in screenshots/.

Monitoring covers service behavior, not drift or clinical performance. Authentication, TLS termination, production alerting and automated retraining are not implemented. Anonymous Grafana viewing and local port-forwards are demonstration settings.

10 Reproduction, access and submission links

Prerequisites: Git, Python 3.12/3.13, Docker with Compose, and Minikube. Start Docker. Allow at least two CPUs and 3 GB for the Minikube cluster plus host resources. Internet is needed for dependencies, source data and images. The private Prayogshala helper is not needed; run the repository commands below.

```
git clone https://github.com/2025ae05588/ml-assignment1.git
cd ml-assignment1
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
python scripts/download_data.py
python scripts/eda.py
ruff check .
pytest -q
python scripts/train.py
bash scripts/docker_verify.sh
bash scripts/deploy_local.sh
```

For local access, run each port-forward in its own terminal from the repository root. In each terminal first export KUBECONFIG="$PWD/deployment/kubeconfig". Then use:

```
minikube -p heart-prayogshala kubectl -- -n heart-mlops \
  port-forward service/heart-api 18002:80
minikube -p heart-prayogshala kubectl -- -n heart-mlops \
  port-forward service/prometheus 19090:9090
minikube -p heart-prayogshala kubectl -- -n heart-mlops \
  port-forward service/grafana 13000:3000
minikube -p heart-prayogshala kubectl -- -n ingress-nginx \
  port-forward service/ingress-nginx-controller 18080:80
```

VM-local URL	Use
http://127.0.0.1:18002/docs	Interactive JSON prediction
http://127.0.0.1:18080/health	Ingress verification
http://127.0.0.1:19090/targets	Prometheus target health
http://127.0.0.1:13000/d/heart-api/heart-api-operations	Grafana dashboard

Local URLs refer to the machine running the forwards; no public API URL is claimed. Keep forwards running while viewing. Stop with Ctrl+C; stop the cluster with minikube -p heart-prayogshala stop. MLflow startup is on page 6.

Repository and full setup instructions

Video walkthrough on GitHub - MP4, approximately 4.98 MB

If GitHub does not play the video inline, use Download raw file on its file page. The repository also contains code, cleaned data, notebook/scripts, model and metrics, tests, workflow, Dockerfile, manifests, screenshots and this report.

Reference: scikit-learn Pipeline

Reference: MLflow tracking